TECHNION
Israel Institute
of Technology

Computer
Science
Department

Operating Systems - 02340123

Homework Exercise 1 – Wet

Spring 2026

Teaching Assistant in charge:

Tsvika Lazar

Assignment Subjects & Relevant Course material:

- **Processes and IPC (inter-process communication)**
- **Recitations 1-3 & Lectures 1-3**
- **HW0**

Introduction

A Unix shell is a command-line interpreter or shell that provides a command-line user interface for Unix-like operating systems[1].

There are several well-known shells on Unix systems, with Bash being one of the most widely used. Bash is a command processor that typically runs in a text window, allowing users to interact by typing commands that trigger various actions.

In this assignment, you will implement a 'smash' (small shell) that mimics the behavior of a real Linux shell but supports only a limited subset of Linux shell commands.

Be sure to review the entire document before you start working or asking questions.

Description

In the skeleton_smash.zip file, you will find the following files:

- Commands.cpp – base code for handling commands.
- smash.cpp – the main shell source code.
- Makefile – for compiling the project.

You must complete the code in Commands.cpp and add signal handling. The functions that define and handle the signal routines will be placed in signals.cpp.

The program will function as follows:

- The program waits for commands typed by the user and executes them.
- It can execute a limited number of **built-in commands**, which will be listed below.
- When the program receives a command that is not one of the **built-in commands** (referred to as an **external command**), it attempts to run it like a normal shell. The method for running external commands will be described later.
- Whenever an error occurs, an appropriate error message should be printed, and the program should return to parsing and executing the next command.
- If an empty command is entered, it will be ignored, and the prompt will be shown again on a new line.
- While waiting for the next command to be entered, the program should display the following prompt (this text can be changed using the **chprompt** command):

```
smash>
```

Assumptions

You may assume the following:

- Each command appears on a separate line and cannot exceed **200** characters. (You may not assume a maximum length for individual parameters).
- The number of arguments per command is up to **20**, including the command name itself.
- Any number of spaces (see WHITESPACE macro in `Commands.h`) can be used between words in the same command line and at the beginning of the line, unless stated otherwise.
- Your smash should support up to **100** processes running simultaneously.
- File and folder names do not include special characters and are in lowercase.
- I/O redirection and Pipe commands are correctly formatted.
- For this assignment, `new` or `malloc` won't fail, and memory leaks won't be checked.

[1] https://en.wikipedia.org/wiki/Unix_shell

Detailed Description

As mentioned before, there are two main types of commands in smash: **built-in commands** and **external commands**. (**Special commands** will be described later).

Built-in commands are features provided by the shell for its users. They run within the same process as the shell and should not be forked and executed separately. Generally, built-in commands are simple and do not execute external executables to achieve their goals. Instead, they usually perform simple system calls or maintain internal shell data structures.

External commands usually require running an external executable. To execute these, the shell runs them in a child process by calling fork() and execv()cd afterwards.

What are jobs?

In shell terminology, a job is a process managed by the shell (processes forked by the shell process). Each job has its own job ID, which is assigned by the shell once it's inserted into the jobs list and remains unchanged. Additionally, each job has a process ID (PID) assigned by the kernel. **Note that the job ID, assigned by the shell, is different from the process ID (PID) assigned by the kernel.**

A shell job can be in one of two states:

1) **Foreground**:

   When you type a command in the shell terminal window, the command will start running and cause the shell to be **blocked**, waiting for the command to finish (the shell waits for the [child] process to finish in this case). This is called a foreground job, where the shell waits for the process to complete before accepting new commands.

2) **Background**:

   When an ampersand symbol `&` is typed at the end of a command line, it means that the shell should run this command in the background. This means the job will not occupy the terminal window, and the shell prompt will be displayed immediately, allowing users to type new commands even though the background job is still running. The shell runs the job without waiting for its completion.

What is the jobs list?

Any job that is sent to the background (using the ampersand symbol `&`) is added to the **jobs list**, allowing users to query and manage them later by their associated **job ID**. For example, the user can ask the shell to print all the jobs it controls using the `jobs` command.

Jobs can be removed from the **jobs list** by the `fg` command, which brings a job from the background to the foreground, or because the job is finished. Refer to the `fg` and `jobs` commands, and signal handling for more details on managing the jobs list.

Deleting Finished Jobs:

Finished jobs should be deleted from the **jobs list** (1) before executing any command, (2) before printing the **jobs list** (jobs command), and (3) before adding new jobs to the **jobs list**.

Assigning job ID:

A job receives its **job ID** upon its first insertion into the **jobs list**, which remains unchanged. The **job ID** is assigned as follows: `Job ID = maximal job ID in jobs list + 1` If the list is empty, then `Job ID = 1`.

Built-in commands

Your smash should support and implement a limited number of shell commands (features). These commands should be executed from the smash process itself, meaning you should not fork the smash process to run them. Instead, they should be run directly from the smash code.

Below are the details of the built-in commands that your smash should support:

1. chprompt command (warm-up)

Command format:

```
chprompt  [new-prompt]
```

Description:

chprompt command will allow the user to change the prompt displayed by the smash while waiting for the next command.

If no parameters are provided, the prompt shall be reset to smash. If more than one parameter is provided, only the first parameter will be used, and the rest will be ignored.

Note: This command will not change the prompt in error messages or the output for showpid (covered later).

Example:

```
smash> chprompt Jeffry_Epstein_didn't_kill_himself
Jeffry_Epstein_didn't_kill_himself> chprompt
smash>
```

2. showpid command

Command format:

```
showpid
```

Description:

showpid command prints the smash PID.

Example:

```
smash> showpid
smash pid is 30903
smash>
```

Error handling:

If any number of arguments were provided with this command, they will be ignored.

3. pwd command

Command format:

```
pwd
```

Description:

pwd command has no arguments.

The pwd command prints the full path of the current working directory. In the next section, we will learn how to change the current working directory using the `cd` command.

Hint: You may use getcwd system call to retrieve the current working directory.

Example:

```
smash> pwd
/home/ayalafos/234123/homeworks/smash/build
smash>
```

Error handling:

If any number of arguments were provided with this command, they will be ignored.

4. **cd command**

Command format:

```
cd <new-path>
```

Description:

The cd (Change Directory) command takes a single argument <path> that specifies either a relative or full path to change the current working directory to.

There's a special argument, `-`, that cd can accept. When `-` is the only argument provided to the cd command, it instructs the shell to change the current working directory to the last working directory.

For example, if the current working directory is X, and then the cd command is used to change the directory to Y, executing `cd -` would then set the current working directory back to X, executing it again would set the current directory to Y.

Hint: You may use chdir system call to change the current working directory.

Note: If no argument is provided, this command has no impact.

Example:

```
smash> cd -
smash error: cd: OLDPWD not set
smash> cd dir1 dir2
smash error: cd: too many arguments
smash> cd /home/ayalafos
smash> pwd
/home/ayalafos
smash> cd ..
smash> pwd
/home
smash> cd -
smash> pwd
/home/ayalafos
smash>
```

Error handling:

If more than one argument was provided, then cd command should print the following error message:

```
smash error: cd: too many arguments
```

If the last working directory is empty and `cd -` was called (before calling cd with some path to change current working directory to it) then it should print the following error message:

```
smash error: cd: OLDPWD not set
```

If chdir() system call fails (e.g., <path> argument points to a non-existing path) then perror should be used to print a proper error message (as described in **Error Handling** section).

5. **jobs command**

Command format:

```
jobs
```

Description:

jobs command prints the **jobs list** which contains the unfinished jobs (Those running in the background).

The list should be printed in the following format: [<job-id>] <command>, where <command> is the original command provided by the user (including spaces, and aliases as discussed later).

The jobs list should be printed in a sorted order w.r.t the job-id.

Note: Make sure you delete all finished jobs before printing the jobs list.

Note: sleep is an external command, we'll see external commands in the next section.

Example:

```
smash> sleep 100&
smash> sleep 200&
smash> jobs
[1] sleep 100&
[2] sleep 200&
smash>
```

Error handling:

If any number of arguments were provided with this command, they will be ignored.

6. **fg command**

Command format:

```
fg [job-id]
```

Description:

fg command brings a process that runs in the background to the foreground.

fg command prints the command line of that job along with its PID (as can be seen in the example) and then waits for it (hint: waitpid), which in effect will bring the requested process to run in the foreground.

The job-id argument is an optional argument. If it is specified, then the specific job which its job id (as printed in jobs command) should be brought to the foreground. If the job-id argument is not specified, then the job with the maximal job id in the jobs list should be selected to be brought to the foreground.

Side effects: After bringing the job to foreground, it should be removed from jobs list.

Example:

```
smash> sleep 100&
smash> sleep 200&
smash> sleep 500&
smash> jobs
[1] sleep 100&
[2] sleep 200&
[3] sleep 500&
smash> fg 3
sleep 500& 901
```

Error handling:
If the syntax (number of arguments or the format of the arguments) is invalid, then an error message should be printed as follows:

```
smash error: fg: invalid arguments
```

If job-id was specified with a job id that does not exist, then the following error message should be reported:

```
smash error: fg: job-id <job-id> does not exist
```

If fg was typed with no arguments (without job-id) but the jobs list is empty, then the following error message should be reported:

```
smash error: fg: jobs list is empty
```

7. **quit command**

Command format:

```
quit [kill]
```

Description:
quit command exits the smash. Only if the kill argument was specified (which is optional), then smash should kill (by sending SIGKILL signal) all of its unfinished jobs and print, before exiting, the number of processes/jobs that were killed, their PIDs and command-lines.

Note: You may assume that the kill argument, if present, will appear first.

Note: When using quit kill, if no jobs remain, you must still print:

```
smash: sending SIGKILL signal to 0 jobs:
```

Example:

```
smash> quit kill
smash: sending SIGKILL signal to 3 jobs:
30959: sleep 100&
30960: sleep 200&
30961: sleep 10&
Linux-shell:
```

Error handling:

If any number of arguments (other than kill) were provided with this command, they will be ignored.

8. **Kill command**

Command format:

```
kill -<signum> <jobid>
```

Description:
Kill command sends a signal whose number is specified by <signum> to a job whose sequence ID in jobs list is <job-id> (same as job-id in jobs command) and prints a message reporting that the specified signal was sent to the specified job (see example below).

Example:

```
smash> kill -9 1
signal number 9 was sent to pid 30985
smash>
```

Error handling:
If the syntax (number of arguments or the format of the arguments) is invalid, then an error message should be printed as follows:

```
smash error: kill: invalid arguments
```

If job-id was specified with a job id which does not exist, then the following error message should be reported:

```
smash error: kill: job-id <job-id> does not exist
```

If the kill system call fails (includes cases where the signal number falls outside the expected range), then report the failure using `perror` (as described in **Error Handling** section).

9. alias command

Command format:

```
alias <name>='<command>'
```

Description:

The alias command is used to create an alias, which is a shortcut that allows a string to be substituted for a command. This can include the command itself and its parameters, but it cannot replace parameters alone.

Where <name> is a case-sensitive, user-defined string that can include only letters (a-z, A-Z), numbers (0-9), and underscores (_). It must not be an internal reserved keyword of the shell (eg., `quit`, `jobs` etc…)

If no arguments are provided, list all current aliases in the order they were created, each on separate line.

Note: an alias can also appear within **Special Commands**, which we will discuss later.

Note: When replacing aliases, only match the first word, until a space or special character like a pipe | appears (as we will discuss later).

Note: If the input is not an alias command, you may assume that whatever comes before >, |, etc., is the command.

Example:

```
smash> alias ll='ls -l'
smash> ll
total 0
-rw-r--r-- 1 user user 0 Jan 1 00:00 file.txt
smash> alias s100='sleep   100  '
smash> alias
ll='ls -l'
s100='sleep   100  '
smash> alias s='sleep'
smash> s 3
```

After 3 seconds:

```
smash>
```

Error handling:
If the syntax is invalid (Including non-legal <name>), then an error message should be printed as follows:

```
smash error: alias: invalid alias format
```

If the alias name conflicts with an existing alias or a reserved keyword, then the following error message should be reported:

```
smash error: alias: <name> already exists or is a reserved command
```

Notes:

- No need to check if <command> is legal.
- You may use the following regex expression **over the stripped command** to validate format: ^alias [a-zA-Z0-9_]+='[^']*'$
- Recursive aliases will not be tested.

10. unalias command

Command format:

```
unalias <name_1> <name_2> …
```

Description:

The unalias command removes the specified aliases separated by spaces.

Example:

```
smash> alias ll='ls -l'
smash> unalias ll
smash>
```

Error handling:
If one or more of the provided alias names do not exist, the deletion process stops **at the first invalid occurrence**, then the following error message is reported:

```
smash error: unalias: <name> alias does not exist
```

where <name> is the name of the first non-existing alias.
If no arguments are provided, then an error message should be printed as follows:

```
smash error: unalias: not enough arguments
```

11. unsetenv command

Command format:

```
unsetenv <variable_1> <variable_2> …
```

Description:

The unsetenv command removes the specified environment variables separated by spaces. After successful execution, the specified variables will no longer exist in the environment.

Note: To check if an environment variable exists, open /proc/<pid>/environ and manually scan the NUL-separated entries. **You are not allowed** to check it by iterating over the entries in the environ variable.

Note: To remove an environment variable, directly modify the char **__environ array.

Note: You do not need to support adding new environment variables, only removing them.

Note: You may assume that once a variable is **successfully** unset, it will not be unset again.

Example:

```
smash> unsetenv TMPDIR
smash>
```

Error handling:
If one or more of the provided environment variable names do not exist, the deletion process stops at the first invalid occurrence, and the following error message is reported:

```
smash error: unsetenv: <variable> does not exist
```

where <variable> is the name of the first non-existing environment variable encountered.
If no arguments are provided, then an error message should be printed as follows:

```
smash error: unsetenv: not enough arguments
```



What are Environment Variables?

Environment variables are key-value pairs stored by the shell that affect the way running processes behave. They're commonly used to store:

- Configuration settings (like PATH, HOME, TMPDIR)
- Temporary data
- User preferences or script behavior toggles

Testing with Environment Variables:

You can create a temporary environment variable in your shell (for testing) like this:

```
export MYVAR=HelloWorld
```

to verify that it's set use:

```
echo $MYVAR
```

to remove it manually use:

```
unset MYVAR
```

12. sysinfo command

Command format:

```
sysinfo
```

Description:

The sysinfo command prints general system information retrieved directly from the kernel. This includes the following details: Operating system name, Hostname, Kernel release and version, Machine architecture, and system boot time.

Example:

```
smash> sysinfo
System: Linux
Hostname: ubuntu-smash-vm
Kernel: 6.8.0-31-generic
Architecture: x86_64
Boot Time: 2025-10-18 17:42:31
smash>
```

Error handling:

If any number of arguments were provided with this command, they will be ignored.

Built-in commands in background:

In this environment, built-in commands should disregard the `&` symbol, so running a command with `&` to send it to the background won't work with built-in commands. This means that commands like `pwd&` and `pwd` would have **the same effect**.

Note: We will not test **built-in commands** that accept optional parameters using the `[]` notation if the `&` symbol can be used as a valid input to replace the optional parameter. For example, this applies to the `chprompt &` command.

External commands

Besides the built-in commands, the smash should support executing external commands that are not part of the built-in commands. External command is any command that is not a built-in command or a special command.

The smash should execute the external command and wait until the external command is executed.

Command line:

```
<command> [arguments]
```

Where `command` is the name of the external command/executable.
The arguments are optional, i.e., if they exist in the external command line then they should be passed as arguments of the external command. (**Reminder:** you may assume that the number of arguments is up to **20**).

How to run:

1) Simple external command: The external command can be provided as a simple command to run some executable with its arguments, for example:

```
a.out arg1 arg2
```

For this type of external commands, you **MUST** run them explicitly using one of the exec family of system calls. In other words, you are not allowed to run these commands by launching an instance of bash and feeding it the command. Note that all the commands you know should work using this mechanism (i.e. sleep, ls,…) as long as they are "simple external commands"

Hint: Consider using a function that searches for and executes commands from the PATH environment variable to handle both executable commands and file paths.

2) Complex external command: On the other hand, the external command could be provided in a complex way, i.e., with the use of one of the following special characters (wildcards): "*" and "?".
For example:

```
rm *.txt
```

For this type of external commands, instead of loading and running the given binary itself, you **MUST** run the external command using bash, which already has the mechanism to parse those complex commands and execute them. This can be done by executing bash (using one of the exec family of functions) with your received <command> [args] in the following form:

```
bash -c "<command> [args]"
```

For example:

```
bash -c "rm *.txt"
```

In this case, smash will run a new instance of bash process while passing ["-c", "rm *.txt"] as the command line arguments to the new bash process. The new instance of bash will run the complex-external-command transparently to you, which will first parse the given command "rm *.txt" and then execute it in a forked child.

Note: Any external command that contains one of the following special characters ["*", "?"] should be considered as a complex external-command and executed through a new instance of bash as explained above.

Note: Running bash means calling one of the exec family of functions on "/bin/bash" binary, namely, "/bin/bash" should be provided to the exec system call as the path argument and ["-c", "complex-external-command"] should be provided as the arguments list to the bash binary, which is about to be executed.

Note: No, you don't have to learn bash language for this assignment.

External commands in background:

External commands can be executed in the background as in most Linux shells.

If a “&” sign is added to the end of a command line, then this command should be executed in the background.

For example:

```
ls -l &
```

When an external command ends with “&” (ignoring white spaces around it) it should be executed as explained above (in the external commands section) with a minor change that smash should not wait for the command completion.

The command being executed in the background should be added to the **Jobs list** (as we saw `sleep&` example in the jobs command).

Note: The “&” may or may not have white spaces around it.

Special commands

In this section, there is one bonus question. Your maximum grade for this assignment can be 110 points.

1) IO redirection

Your smash code should support simple IO redirection. You can assume, for simplicity, that each typed command could have up to one character of IO redirection followed by a string. IO redirection characters that your smash should support: `>` and `>>`.

How to use redirection features:

- `command **>** output-file`: using the `**>**` redirection character (as in this example) causes the command stdout file-descriptor to be redirected to output-file, and writes any output produced by `command` to the given output file. If the output file does not exist then it creates it, and if it exists then it **overrides** its content.
- `command **>>** output-file`: the same as `>` except that using `>>` will **append** command output to the given output file if it exists. If the output-file does not exist, then `>` and `>>` will produce the same result.

Note: To make the implementation easier for you, your I/O redirection commands won't be tested with commands whose implementation includes the wait system call.
Note: Unlike the real shell, commands containing I\O directions should ignore the `&` symbol, and they cannot be killed (they will not be tested for receiving signals). This is done for simplification purposes only; a real shell should be able to handle those things.

Example:

```
smash> showpid > pid.txt
smash> cat pid.txt
smash pid is 27882
smash> showpid >> pid.txt
smash> cat pid.txt
smash pid is 27882
smash pid is 27882
smash> ls -l > ls.txt
smash>
```

2) Pipes

You can add pipe functionality to your `smash`. This should work similarly to redirection commands. The pipe characters that your `smash` should support are `|` and `|&`. You may assume only one type of pipe will appear in each command with a single instance.

How to use pipe features:

- `command1 | command2`: Using the pipe character `|` will produce a pipe that redirects `command1`'s `stdout` to its write channel and `command2`'s `stdin` to its read channel.
- `command1 |& command2`: Using the pipe character `|&` will produce a pipe that redirects `command1`'s `stderr` to the pipe's write channel and `command2`'s `stdin` to the pipe's read channel.

Note: As with I/O redirection, pipe commands will be tested with simple commands, won't be run in the background, and won't be sent signals.

Note: A command won't include both I/O redirection and a pipe.

Example:

```
smash> cat ls.txt
total 140
-rwxrwxrwx 1 root root 6376 Nov 12 10:52 Makefile
-rwxrwxrwx 1 root root 146 Nov 12 14:11 ls.txt
-rwxrwxrwx 1 root root 38 Nov 12 14:11 pid.txt
-rwxrwxrwx 1 root root 82832 Nov 12 13:41 smash
smash> cat ls.txt | grep Makefile
-rwxrwxrwx 1 root root 6376 Nov 12 10:52 Makefile
smash>
```

3) du command

Command format:

```
du [directory-path]
```

Description:

The du command recursively calculates and displays the total disk space usage for the specified directory in whole kilobytes (KB), rounding up as necessary. If no path is given, the current working directory is used.

Note: Symbolic links (symlinks) must not be followed during traversal.

Note: You may assume that any provided path does exist.

Example:

```
smash> du my_folder
Total disk usage: 4856 KB
smash>
```

Error handling:

If more than one argument was provided, the following error message should be printed:

```
smash error: du: too many arguments
```

4) whoami command

Command format:

```
whoami
```

Description:

The whoami command displays the current user's username, user ID (UID), group ID (GID), and the user's home directory, in this specific order.

Note: You may Not assume that the home directory and the username share the same name.

Example:

```
smash> whoami
johndoe
1000
1000
/home/johndoe
```

Error handling:

If any number of arguments were provided with this command, they will be ignored.

5) usbinfo command (Bonus 10 Points)

Command format:

```
usbinfo
```

Description:

The usbinfo command displays detailed information about all currently connected USB devices recognized by the kernel.

Each device entry includes its kernel-assigned device number, vendor ID, product ID, manufacturer, product name, and maximum power consumption.

Devices should be printed sorted by device number, in ascending order using this format:

Device <devnum>: ID <vendor>:<product> <manufacturer> <product_name> MaxPower: <power>mA

Note: If a field is unavailable, print N/A in its place.

Note: You may not use any external utilities to generate the output.

Example:

```
smash> usbinfo
Device 1: ID 0781:5583 SanDisk Ultra Fit  MaxPower: 200mA
Device 2: ID 046d:c52b Logitech N/A  MaxPower: 98mA
smash>
```

Error handling:

If any number of arguments were provided with this command, they will be ignored.

If no USB devices are detected, print:

```
smash error: usbinfo: no USB devices found
```

Handling signals

Your smash should support catching Ctrl+C signals and handle them as will be explained.

Ctrl+C causes the shell to send SIGINT to the process running in the foreground. Note that from the Linux shell's perspective, the smash is the process running in the foreground, not the process currently running within your smash. Therefore, SIGINT is supposed to be sent to the smash's main process.

Your smash should route this signal to the running process in the foreground. If there is no process running in the foreground, then your smash should ignore it.

When Ctrl+C is pressed, your smash should perform the following actions:

- Print the following message:
  `smash: got ctrl-C`
- Send SIGKILL to the process in the foreground. If no process is running in the foreground, then no signal will be sent.
- Print the following information:
  `smash: process <foreground-PID> was killed`

Example:

```
smash> sleep 1000
^Csmash: got ctrl-C
smash: process 31951 was killed
smash> sleep 1000
smash>
```

Important Notes

setpgrp

Please note that you need to use the setpgrp system call to change the group ID of all your forked children. This is necessary because the Linux shell may send SIGINT (in case of Ctrl+C) to your smash and all its children. This can happen because the Linux shell sends signals to all processes that share the same group ID.

When you call `fork`, the created process (the child process) will have the same group ID as its parent unless you change it through the `setpgrp` system call. You need to call `setpgrp` right after `fork` in the child code. For example:

```
pid = fork();
if (pid == 0) {
    // Child process code goes here
    setpgrp();
    ...
} else {
    ...
}
```

Error Handling

- If multiple error messages are applicable to the occurred error (e.g., entering the command `fg 8 a` with fewer than 8 jobs), print the first error message specified in the instructions.
- If a system call fails, your smash should use the `perror` function to report the failure. The error message (to be provided to `perror`) should be as follows:

  ```
  smash error: <syscall name> failed
  ```

  Where `syscall name` is the name of the called method to execute the system call. For example, if a call to `fork` failed, it should report the failure this way:

  ```
  perror("smash error: fork failed");
  ```

- **All error messages printed in red throughout this document should be printed to `cerr` and NOT `cout`.**

Important Notes and Tips

- The assignment will be graded by automated tests. Write your own tests to ensure that your system works according to the specifications provided.
- **Pay close attention to the print formats specified throughout the assignment. The tests are automated, and missing a space in the output can cost you points.**
- Use the same setup required for HW0.
- First, try to understand exactly what your goal is.
- Determine which data structures will serve you in the easiest and simplest way. Feel free to use any library for algorithms or data structures, including `std::vector`, `std::list` and `std::string`.
- Do not use `fstream`, `ofstream`, `opendir`, `readdir`, `closedir`, or any similar libraries that allow bypassing the implementation with basic syscalls. **Failure to follow this guideline will result in point deductions.**
- The given skeleton code serves as a suggestion, you're not obligated to follow it, but you must write your solution in C/C++ and submit a Makefile with it. If you do not use the given skeleton, ensure that your Makefile contains the rule `make smash`, which compiles all your files into an executable. **Code that does not follow these rules, will not receive a grade!**

  We sincerely promise that there is no need to add more files or change the Makefile to complete the assignment.
- Make sure to write your IDs in the provided Makefile.
- Start working on the assignment ASAP. The deadline is final.
- Using AI tools (e.g., GPT, or similar) to copy or generate code sections is strictly prohibited.
- **Any form of academic dishonesty will result in strict disciplinary action.**

Questions & Answers

- The Q&A for the exercise will take place at a public forum Piazza only. Please do not send questions to the private email addresses of the TAs.
- Critical updates about the HW will be published in pinned notes in the piazza forum. **These notes are mandatory, and it is your responsibility to be updated.**

A number of guidelines to use the forum:

- Read previous Q&A carefully before asking a question; **repeated questions will go without answers**, to prevent contradictions and ensure easy follow up by your classmates.
- When writing in Hebrew please choose 'Right to Left' so that your question is readable.
- Be polite, remember that course staff does this as a service for the students.
- You're not allowed to post any kind of solution and/or source code in the forum as a hint for other students; In case you feel that you must discuss such a matter, please come to the reception hours.
- When posting questions regarding this assignment, put them in the **hw1_wet** folder.

Late Days

- Please do not send postponement requests to the private email of the TA responsible for this assignment. If you need a postponement for **legitimate** reasons, fill out this form.
- **The form will close 24 hours before the submission deadline**, so make sure to submit your request in advance – do not wait until the last minute.

Re-Submissions

- Students who received a grade of 90 or higher are not eligible for resubmission.
- Each resubmission will result in a 20-point deduction, **with no exceptions**.
- You may modify up to 5 lines in total.
- The above rules are subject to change at the discretion of the course staff.

Appeals

- Appeals will be accepted only in justified cases, such as a widespread issue affecting multiple students or a unique, exceptional circumstance affecting an individual.
- **All** appeals must be submitted through the course representative.
- Direct appeals sent to the TA in charge of this assignment **will not** be accepted.

FAQs

1. Is the virtual machine for OS course the same as the one used in ATAM?
   A: No, they are different. We want you to use a specific Ubuntu version.

2. What is considered a reserved keyword?
   A: Only built-in and special commands are reserved keywords.

3. Where should I write and execute my code?
   A: You can write your code locally, but you must compile, run and test it inside your VM to ensure everything works correctly. We will evaluate your program in a similar environment.

4. Which exec* function should we use?
   A: You can use any execv, execlp, execvp, execvpe, execve, execl, or execle. Your output will be normalized accordingly during the tests.

5. Can I modify the provided file X or change the implementation of function/class in that file?
   A: Yes, they're only provided to help you get started, you may change or re-implement them.

6. Should we handle arguments enclosed in single or double quotes?
   A: No, there's no need to worry about that.

7. What is the maximum length of the full path for the current working directory?
   A: It depends on the system and is defined by the constant PATH_MAX.

8. You mentioned that any number of spaces can appear between arguments, but the regex for the alias command is different. Is that a mistake?
   A: No, that's intentional, it's just meant to make the implementation simpler for you.

9. Can you clarify when we should use fork() and when we shouldn't?
   A: You should use fork() only when it's needed. For example, using fork() to implement showpid would be incorrect, unless you call getppid(). Similar issues arise with other commands as well.

10. Are we allowed to use external Bash commands or system utilities (like ps, or ifconfig) inside our built-in commands?
    A: No, you shouldn't use them. Implement the required functionality using system calls instead.

11. Does an argument like 01 (or any number with leading zeros) count as valid and should it be interpreted as 1? And will negative numbers be tested as well?
    A: These cases won't be tested.

12. Is there a specific permission we should set for a new file created through I/O redirection?
    A: Using permission 0x666 (read and write for all) is fine.

13. Can you clarify which functions are allowed or forbidden to use in the assignment?
    A: There's no complete list of allowed/forbidden functions, yet here's a guiding principle;
    If a function abstracts away logic you are expected to implement (e.g., getpwuid() in whoami) then it's not allowed. If it is a utility for basic functionality (like printing) then it's allowed.

14. Are we expected to support running smash as an external command from within smash itself?
    A: No, one smash is enough chaos for this assignment. Trust us, you don't want recursive smash.

Submission Instructions

You should create a zip file (use zip only, not gzip, tar, rar, 7z or anything else) containing the following files:

1) All source files you wrote with no subdirectories of any kind.
2) A file named submitters.txt which includes the ID, name and email of the participating students. Use the following format:

```
Linus Torvalds linus@gmail.com 234567890
Ken Thompson ken@belllabs.com 345678901
```

Important Notes:

- Ensure that the zlsip file structure is exactly as outlined. The zip should contain only the specified files, without directories.

  You can create the ZIP file (as recommended) by running:

  ```
  make submit
  ```

  If everything is correct, you should see the following messages:

  ```
  ✔ All required files found. Creating submission archive...
  Created: <ID1>_<ID2>.zip
  ```

  The zip should look as follows:

  ```
  zipfile -+                  |
  Commands.h
  Commands.cpp
  signals.h
  signals.cpp
  smash.cpp
  Makefile
  submitters.txt
                  |
                  +- other files
  ```

- To run the basic tests, execute the following command:

  ```
  make test
  ```

- When you submit, **retain your confirmation code and a copy of the file(s)**, in case of a technical failure. Your confirmation code is **the only valid proof** that you submitted your assignment when you did.

Good luck and have fun!

- The Course Staff